

Architecture & Implementation Plan: SyncSphere

(Cloud-Native Collaboration Workspace)

Executive Summary & Problem Statement

Problem Statement

In modern software and research engineering, teams rely on fragmented tools for document collaboration, file storage, task tracking, and artifact processing. Existing commercial tools (like basic Jira or GitHub clones) operate as static CRUD monoliths, missing cloud-native capabilities such as:

- 1. **Real-Time Asynchronous Artifact Processing:** Automatic scanning of uploaded code/document assets for leaked credentials, malware signatures, and automatic metadata extraction.
- 2. **Cloud Observability & Governance:** Integrated telemetry, latency tracking, and audit logging for security compliance.
- 3. **Budget-Aware Elastic Scaling:** Microservice design leveraging Azure's serverless pay-as-you-go and free tier quotas.

Project Title

SyncSphere — *Event-Driven Cloud-Native Workspace for Collaborative Engineering & Artifact Intelligence*

Real-Life Significance & Resume Value

Unlike standard CRUD web apps, **SyncSphere** demonstrates enterprise-grade Cloud Architecture principles:

- **Microservices & Serverless Decoupling** (Azure App Service + Azure Functions + Azure Event Grid)
- **Event-Driven Architecture** (Async Blob storage triggers -> Function processing)
- **Zero-Trust Security & RBAC** (Azure Entra ID / JWT + Azure Key Vault + Managed Identities)
- **Cloud Observability** (Azure Application Insights + Log Analytics)
- **Production-Grade DevOps** (Bicep/ARM IaC + GitHub Actions CI/CD)

Course Rubric & Syllabus Alignment (23CSE363 - 20 Marks Total)

Rubric Criteria	Weight	How SyncSphere Fulfills It
1. Objective & Requirements	2	Clear problem statement, documented requirements, real-life enterprise utility.
2. System Architecture & Design	4	Multi-tier cloud architecture (Frontend, REST API microservices, Event-Driven Serverless, Cosmos DB/Blob Storage).
3. Implementation & Functionality	4	Fully working React + Node.js/Python FastAPI app hosted on Azure, executing live collaboration & artifact processing.
4. Security & Access Control	2	Role-Based Access Control (RBAC), JWT authentication, Key Vault secret management, HTTPS/TLS 1.3 encryption.
5. Database & Data Management	2	Dual-storage paradigm: Azure Cosmos DB (Document model for tasks/workspaces) + Azure Blob Storage (Object store for assets).
6. Deployment & DevOps	2	Infrastructure as Code (Bicep/ARM) + Automated GitHub Actions CI/CD pipeline deploying to Azure.
7. Monitoring & Performance	1	Azure Application Insights integration for distributed tracing, CPU/Memory telemetry, and custom logs.

8. Documentation & Presentation	2	Architecture diagrams, comprehensive API docs, step-by-step verification, and deployment guide.
9. Innovation & Problem Solving	1	Automated secret/leak detection on uploaded files via Serverless Azure Functions + AI summary integration.

System Architecture & Technology Stack

```
graph TD
    User([User / Browser]) -->|HTTPS / WSS| SWA[Azure Static Web Apps\nReact Frontend]
    SWA -->|REST API| API[Azure App Service / Container App\nBackend Microservice API]
    API -->|Auth & RBAC| Auth[Azure Entra ID / JWT]

    API -->|CRUD Data| DB[(Azure Cosmos DB\nNoSQL Database)]
    API -->|Upload Assets| Storage[(Azure Blob Storage\nProject Assets)]

    Storage -->|BlobCreated Event| EG[Azure Event Grid / Service Bus]
    EG -->|Async Trigger| Func[Azure Functions\nServerless Artifact Processor]

    Func -->|Security Scan & Metadata| DB
    Func -->|Secrets Management| KV[Azure Key Vault]

    API -->|Telemetry & Logs| AppInsights[Azure Application Insights & Log Analytics]
    Func -->|Telemetry & Logs| AppInsights
```

Azure Services & Tech Stack (Student Budget Optimized)

- 1. Frontend:
 - Tech: React.js, TailwindCSS / Lucide icons, Vite.
 - Azure Host: Azure Static Web Apps (Free Tier - \$0/mo).
- 2. Backend API:
 - Tech: Node.js (Express) or Python (FastAPI).
 - Azure Host: Azure App Service (F1 Free Tier / B1 Basic - ~\$0-13/mo covered by \$100 credit).
- 3. Async Serverless Processing:
 - Tech: Node.js / Python Azure Functions (Consumption Plan - 1 million free executions/mo).
 - Trigger: Azure Blob Storage Event Grid Trigger (Executes auto-scan upon asset upload).
- 4. Data Management:
 - Database: Azure Cosmos DB (Free Tier: 1,000 RU/s + 25GB Storage forever free).
 - File Storage: Azure Blob Storage (Hot Tier, encrypted with Microsoft Managed Keys).
- 5. Security & Governance:
 - Secrets: Azure Key Vault (Stores DB connection strings, API keys, JWT secrets).
 - Auth: JWT / Azure Entra ID with Role-Based Access Control (Admin, Editor, Viewer).
- 6. Observability & Monitoring:
 - Telemetry: Azure Application Insights + Log Analytics Workspace (Free 5GB/month ingest).
- 7. Infrastructure & DevOps:
 - IaC: Azure Bicep / ARM Templates.
 - CI/CD: GitHub Actions workflows for automated build, test, and Azure deployment.

■ Key Functional Modules & Unique Features

1. **Collaborative Workspace & Kanban / Task Board:**
 - Real-time project boards, task assignment, status pipelines (To-Do, In Progress, Code Review, Done).
2. **Event-Driven Artifact Vault & Automated Secret Scanner (Unique Innovation):**
 - When users upload code snippets, configuration files, or docs into project workspaces, an **Azure Function** automatically triggers in the background:
 - Scans files for exposed API keys (e.g. AWS/Azure keys, JWT tokens).
 - Generates file metadata (hash, size, type, preview snippets).
 - Updates the Cosmos DB record with security status (**CLEAN** vs **WARNING: SECRET DETECTED**).
3. **Real-Time Project Analytics & Health Dashboard:**
 - Visual metrics showing project progress, task distribution, asset count, and security audit logs.
4. **Live Azure Application Insights Observability Panel:**
 - Embedded admin telemetry view showing API response latency, request throughput, dependency maps, and server health.

■■ Verification Plan

Automated Tests

- **Backend API Tests:** Integration tests testing Auth, Workspace CRUD, Task updates, and File upload endpoints.
- **Azure Function Tests:** Unit & Integration tests verifying event payload handling and secret scanning logic.

Manual & Azure Verification Steps

1. **Azure CLI Verification:** `az appservice show`, `az functionapp show`, `az cosmosdb show` verifying live Azure provisioning.
2. **Event-Driven Upload Verification:** Uploading a file containing dummy credentials and verifying Azure Function trigger execution & Cosmos DB update within 2 seconds.
3. **Application Insights Verification:** Triggering 10 HTTP requests and checking live telemetry logs in Azure Portal.

■ Proposed Implementation Phasing

- **Phase 1:** Infrastructure Setup & Azure Service Provisioning via Azure CLI / Bicep.
- **Phase 2:** Backend API Microservices & Cosmos DB Schema Implementation.
- **Phase 3:** Azure Function Serverless Event-Driven Artifact Scanner Setup.
- **Phase 4:** Responsive React Workspace Frontend Development & Azure Static Web Apps Integration.
- **Phase 5:** Security (Key Vault & RBAC) + Observability (App Insights) Integration.
- **Phase 6:** GitHub Actions CI/CD Deployment & End-to-End Walkthrough Documentation.